OOP

UNIT 5 – WORKING WITH OBJECTS

Objects become useful when they can be passed, protected, copied, returned, and combined with other objects.

LEARNING OBJECTIVES

- Passing objects by value, reference, and const reference.
- Use const objects, const parameters, and const member functions correctly.
- Explain when the copy constructor is invoked and how memberwise copying works.
- Return objects from functions and distinguish returned values from references to local objects.
- Initialize data members correctly using constructor initializer lists.
- Define and use static data members and static member functions.
- Explain the meaning and limitations of inline functions.
- Distinguish composition from aggregation and trace construction and destruction order.

WORKING WITH COMPLETE OBJECTS

In the previous units, we learned how to define classes, construct valid objects, and observe how objects are destroyed. We can now create a Song object and use its member functions. The next step is to understand how objects participate in larger programs.

```
Song song("Imagine", "John Lennon", 183);
song.play();
song.print();
```

Real programs rarely keep every operation inside main(). Functions receive objects, inspect them, modify them, create new objects, and return results. Classes also contain objects of other classes. These operations introduce important questions about efficiency, protection, copying, and object lifetime.

PASSING OBJECTS TO FUNCTIONS

A function can receive an object in three common ways: by value, by reference, or by const reference. The parameter declaration determines whether a copy is created and whether the function may modify the original object.

Parameter form	Copy created?	May modify original?	Typical purpose
Song song	Yes	No	Function needs an independent copy
Song& song	No	Yes	Function must modify the original
const Song& song	No	No	Efficient read-only access

PASSING BY VALUE

When an object is passed by value, the function receives a **separate (copied) object** initialized from the argument. Changes made inside the function affect only that copy.

```
void renameSong(Song song) {
    song.setTitle("New Title");
}

int main() {
    Song song("Imagine", "John Lennon", 183);
    renameSong(song);
    song.print();
}
```

The original object remains unchanged. Passing by value is useful when a function intentionally needs its own independent copy, but it may be unnecessarily expensive for large objects.

PASSING BY REFERENCE

A reference parameter is another name for the original object. No new Song object is created.

```
void playSong(Song& song) {
    song.play();
}

int main()
{
    Song song("Imagine", "John Lennon", 183);
    playSong(song);
    cout << song.getPlays() << endl;
}
```

Because song refers to the original object, the change remains after the function returns.

PASSING BY CONST REFERENCE

Many functions need to inspect an object without modifying it. Passing by const reference avoids copying while guaranteeing read-only access.

```
void displaySong(const Song& song)
{
    song.print();
}
```

The const keyword prevents the function from calling non-const member functions or changing the object. For read-only parameters of class type, const reference is usually the preferred choice.

CHOOSING A PARAMETER PASSING METHOD

• Use pass by value when the function needs an independent copy.

For small primitive values such as **int, double, and char**, passing by value is normally appropriate. The efficiency concern applies mainly to class objects and other potentially large values.

• Use pass by reference when the function must modify the original object.

• Use pass by const reference when the function only reads the object and copying is unnecessary.

THE CONST KEYWORD

The keyword const expresses a promise: a value will not be modified through a particular name or operation. This promise allows the compiler to detect accidental changes.

CONST MEMBER FUNCTIONS

A member function that does not modify the object should be marked const after its parameter list.

```
class Song
{
public:
    void print() const;
    int getDuration() const;
    int getPlays() const;
};
void Song::print() const
{
    cout << title << " - " << artist << endl;
}
```

The trailing const is part of the function declaration. It promises that the function will not change ordinary data members of the current object.

CONST OBJECTS

```
const Song song("Imagine", "John Lennon", 183);
```

After an object is declared const, its observable state may not be changed. The compiler therefore permits calls only to member functions that are themselves declared const.

```
song.print();   // legal only if print() is const
song.play();    // illegal: play() changes the object
```

WHY CONST CORRECTNESS MATTERS

Suppose displaySong() receives a const Song&. Inside that function, the parameter is read-only. The compiler will allow only const member functions to be called on it.

```
void displaySong(const Song& song)
{
    song.print();          // legal: print() is const
    cout << song.getDuration() << endl;
    // song.play();        // illegal: play() modifies the object
}
```

Const correctness documents intent, prevents accidental modification, and makes a class usable in read-only contexts.

COMMON CONST MISTAKE

```
void Song::print()
{
    cout << title << endl;
}
```

This function does not change the object, but it is not declared const. As a result, it cannot be called for a const Song or through a const Song&. The correct declaration is void print() const.

COPYING OBJECTS

Passing an object by value revealed that a new object may be created from an existing one. C++ uses a special constructor for this operation: the copy constructor.

```
Song original("Imagine", "John Lennon", 183);

Song copy(original) /*copy constructor*/;
```

The second declaration creates a new Song and initializes it from original.

THE COPY CONSTRUCTOR

A copy constructor receives a const reference to another object of the same class.

```
class Song
{
public:
    Song(const Song& other);
};

Song::Song(const Song& other)
    : title(other.title),
      artist(other.artist),
      duration(other.duration),
      plays(other.plays)
{
}
```

The parameter is a reference so that receiving the source object does not require another copy. It is const because copying should not modify the source.

WHEN IS THE COPY CONSTRUCTOR CALLED?

- When a new object is initialized from an existing object.
- When an object is passed to a function by value.
- When an object is returned by value in cases where the compiler does not eliminate the copy.

```
Song first("Imagine", "John Lennon", 183);
Song second = first;        // copy construction
printSong(first);           // copy if parameter is passed by value
```

RETURNING OBJECTS FROM FUNCTIONS

A function may construct an object and **return** it **by value**.

```
Song createSong( ) {
    Song song("Imagine", "John Lennon", 183);
    return song;
}
int main() {
    Song favorite = createSong();
    favorite.print();
}
//The local object song is valid until the function finishes.
```

COPY CONSTRUCTION IS NOT ASSIGNMENT

```
Song first("Imagine", "John Lennon", 183);
Song second = first;        // creates second: copy constructor

Song third("Halo", "Beyonce", 261);
third = first;              // third already exists: assignment
```

COMPILER-GENERATED COPY CONSTRUCTOR

If a class does not define a copy constructor, the compiler normally provides one. It copies each data member from the source object to the new object. This is called memberwise copying.

```
class Song
{
private:
    string title;
    string artist;
    int duration;
    int plays;
};
```

For this Song class, memberwise copying is sufficient because string manages its own resources and the integer members can be copied directly.

When a class later owns dynamic memory through a raw pointer, simple memberwise copying may create a **shallow copy**. That problem belongs to the Rule of Three and will be addressed in the next unit.

NEVER RETURN A REFERENCE TO A LOCAL OBJECT

```
const Song& createSong()        // wrong!!!!
{
    Song song("Imagine", "John Lennon", 183);
    return song;
}
```

The local object is destroyed when the function ends. Returning a reference to it leaves the caller with a dangling reference. Returning the object by value is safe.

CONSTRUCTOR INITIALIZER LISTS

Earlier **constructors** assigned values inside the constructor body.

```
Song::Song(string title, string artist, int duration) //constructor
{
    this->title = title;
    this->artist = artist;
    this->duration = duration;
    plays = 0;
}
```

However, data members are initialized before the constructor body begins. The statements above assign new values after initialization has already occurred.

INITIALIZER LIST SYNTAX

```
Song::Song(string othertitle, string artist, int duration)
    : title(othertitle),artist(artist),
      duration(duration),  plays(0) {}
```

The initializer list appears after the constructor parameter list and before the constructor body. It directly initializes each member.

INITIALIZATION VS. ASSIGNMENT

Initialization	**Assignment**
Creates the initial value of a member	Replaces a value after the member already exists
Occurs before the constructor body	Occurs inside the constructor body or later
Required for some member types	Not available for const members and references

WHEN INITIALIZER LISTS ARE REQUIRED

- For const data members.
- For reference data members.
- For member objects whose class has no default constructor.
- For base classes, which will be studied later.

```
class Track
{
private:
    const int id;
    string& libraryName;
public:
    Track(int id, string& libraryName) : id(id), libraryName(libraryName)
    {    }
};
```

INITIALIZATION ORDER

Members are initialized in the order in which they are declared inside the class, not in the order written in the initializer list.

```
class Song
{
private:
    string title;
    int duration;

public:
    Song(string title, int duration)
        : duration(duration), title(title) { }
};
```

Even though duration appears first in the initializer list, title is initialized first because it is declared first. The best practice is to write initializer lists in declaration order.

STATIC MEMBERS

Ordinary data members belong to individual objects. Each Song has its own title, duration, and play count. Sometimes information belongs to the class as a whole rather than to one object.

STATIC DATA MEMBERS

```
class Song {
private:
    static int numberOfSongs;
public:
    Song();
    ~Song();
    static int getNumberOfSongs();
};
```

Only one numberOfSongs variable exists, regardless of how many Song objects are created.

```
int Song::numberOfSongs = 0;

Song::Song()  {    numberOfSongs++; }
Song::~Song() {    numberOfSongs--; }
```

The definition outside the class allocates storage for the static data member.

STATIC MEMBER FUNCTIONS

```
int Song::getNumberOfSongs()
{
    return numberOfSongs;
}
cout << Song::getNumberOfSongs() << endl;
```

A static member function belongs to the class and can be called without a particular object. Because it has no current object, it has no this pointer and cannot directly access non-static data members.

ORDINARY VS. STATIC MEMBERS

Ordinary member	**Static member**
One copy per object	One shared copy per class
Accessed through a particular object	May be accessed through the class name
Member function has a this pointer	Static member function has no this pointer

INLINE FUNCTIONS

A very small function may be declared inline (inside class declaration – header file)

```
inline int Song::getDuration() const
{
    return duration;
}
```

The inline keyword is a request that the compiler may replace a function call with the function body. The compiler is free to ignore this request.

FUNCTIONS DEFINED INSIDE A CLASS

A function defined inside the class definition is implicitly inline.

```
class Song {
public:
    int getDuration() const { return duration;  }
};
```

Inline is most appropriate for very **small**, **frequently used** functions such as getters. Large functions should remain in the source file for readability and compilation efficiency.

RELATIONSHIPS BETWEEN CLASSES

Large object-oriented programs are built from multiple classes that collaborate. Two common has-a relationships are composition and aggregation.

COMPOSITION

Composition represents strong ownership. The contained object is a direct part of the containing object and normally shares its lifetime.

```
class Album
{
private:
    string name;
    Song firstSong;

public:
    Album(string name, const Song& song)
        : name(name), firstSong(song)
    {
    }
};
```

The Album object contains its own Song object. The Song is constructed before the Album constructor body executes and is destroyed automatically after the Album destructor body finishes.

COMPOSITION AND INITIALIZER LISTS

Suppose Song has no default constructor. Album **must explicitly** initialize its contained Song in the initializer list.

```
class Song {
public:
    Song(string title, string artist, int duration);
};

class Album {
private:
    Song openingTrack;
public:
    Album()   : openingTrack("Imagine", "John Lennon", 183){ }
};
```

Attempting to assign openingTrack inside the Album constructor body would be too late. The contained object must already be initialized before the body begins.

CONSTRUCTION AND DESTRUCTION ORDER (WE ALREADY KNOW THAT!!)

```
class Song {
public:
    Song()  { cout << "Song constructor" << endl; }
    ~Song() { cout << "Song destructor" << endl; }
};

class Album
{
private:
    Song song;
public:
    Album()  { cout << "Album constructor" << endl; }
    ~Album() { cout << "Album destructor" << endl; }
};
```

Output:

```
Song constructor
Album constructor
Album destructor
Song destructor
```

The contained object must exist before the containing object can use it. Destruction occurs in the reverse order.

AGGREGATION

Aggregation represents a weaker relationship. One object refers to another object but does not necessarily own it. The related object may exist independently.

```
class Playlist
{
private:
    const Song& featuredSong;

public:
    Playlist(const Song& song)
        : featuredSong(song)
    {
    }
};
```

The Playlist refers to a Song created elsewhere. Destroying the Playlist does not destroy that external Song. The programmer must ensure that the referenced Song remains alive as long as the Playlist uses it.

COMPOSITION VS. AGGREGATION

Composition	**Aggregation**
Strong ownership	Weak or external association
Contained object is stored directly	Related object is usually referenced through a pointer or reference
Lifetimes are closely connected	Related object may outlive the container
Example: Album contains its own Song	Example: Playlist refers to an existing Song

SUMMARY

This unit extended the basic object lifecycle into practical object collaboration. We learned how objects are transferred to functions, protected with const, copied, returned, initialized, shared at class level, and combined with other objects.

- Passing by value creates a separate object.
- Passing by reference allows modification of the original object.
- Passing by const reference provides efficient read-only access.
- Const member functions can be called for const objects.
- The copy constructor initializes a new object from an existing object.
- Returning objects by value is safe; returning references to local objects is not.
- Initializer lists perform true member initialization and are required in several cases.
- Static members belong to the class rather than to individual objects.
- Inline is a compiler-oriented declaration, not a guarantee of faster code.
- Composition expresses ownership; aggregation expresses a weaker relationship.

REVIEW QUESTIONS

1. What happens when an object is passed by value?

2. When should a parameter be declared as a non-const reference?

3. Why is const reference commonly used for read-only object parameters?

4. What promise does a const member function make?

5. Why can a non-const member function not normally be called for a const object?

6. What is the exact parameter type of a typical copy constructor?

7. List three situations in which a copy constructor may be invoked.

8. What is the difference between copy construction and assignment?

9. Why is returning a reference to a local object incorrect?

11. What is the difference between initialization and assignment?

12. When is a constructor initializer list required?

13. In what order are data members initialized?

14. How does a static data member differ from an ordinary data member?

15. Why can a static member function not directly access title or duration?

16. Does inline guarantee that the compiler will eliminate the function call?

17. What is composition?

18. What is aggregation?

19. If Album directly contains Song, which object is constructed first?

20. What lifetime risk exists when aggregation is implemented with a reference?

OOP

UNIT 5.5 – BUILDING A MUSIC LIBRARY

A COMPLETE CASE STUDY USING THE OBJECT-ORIENTED TECHNIQUES INTRODUCED SO FAR.

LEARNING OBJECTIVES

- Design a small application as a collaboration between several classes.
- Assign a clear responsibility to each class before writing its implementation.
- Use constructors, initializer lists, const correctness, references, and copy construction inside one coherent project.
- Model non-owning relationships between independent objects using aggregation.
- Trace object lifetime while a complete application is created, used, and closed.
- Organize a multi-file C++ project using header and source files.

FROM INDIVIDUAL CLASSES TO AN APPLICATION

The previous units introduced the tools needed to define reliable classes and work with complete objects. The next challenge is not another isolated language feature. It is deciding how several classes should cooperate inside one program.

This unit develops a small music library. Every example belongs to the same project. The program grows one class at a time, and each design decision solves a real need in the application. The purpose is to use the material from Units 2–5 in context rather than repeat its definitions.

THE APPLICATION WE WILL BUILD

The application manages songs, albums, playlists, and a music library. These concepts are related, but they do not have the same responsibility.

Class	**Responsibility**
Song	Stores the information and behavior of one song.
Album	Groups existing songs that belong to one album collection.
Playlist	Groups existing songs selected by a user.
MusicLibrary	Coordinates the songs, albums, and playlists known to the application.

The complete project uses fixed-capacity arrays of pointers. The pointers refer to objects created elsewhere; they do not allocate or delete memory. This keeps the focus on object

collaboration and aggregation. Dynamic ownership is intentionally postponed until the next unit.

PROJECT ORGANIZATION

Each class has a header file containing its declaration and a source file containing its implementation. The main program creates the objects and connects them.

```
MusicLibraryProject/
    Song.h
    Song.cpp
    Album.h
    Album.cpp
    Playlist.h
    Playlist.cpp
    MusicLibrary.h
    MusicLibrary.cpp
    main.cpp
```

The complete source code is provided separately from the chapter. The excerpts below show only the portions needed to understand each design step.

STEP 1: BUILDING SONG

Song is the smallest independent object in the application. It owns its title, artist, duration, and play count. Because each Song stores ordinary values and std::string objects, its state belongs entirely to the object.

```
class Song
{
private:
    string title;
    string artist;
    int durationSeconds;
    int playCount;
    static int numberOfSongs;

public:
    Song();
    Song(const string& title,
         const string& artist,
         int durationSeconds);
    Song(const Song& other);
    ~Song();

    const string& getTitle() const { return title; }
```

```
    int getDurationSeconds() const { return durationSeconds; }

    void play();
    void print() const;
    static int getNumberOfSongs();
};
```

The public interface reflects real operations. A song can be played and printed. Its identifying information can be inspected, but not modified directly by outside code.

CONSTRUCTING VALID SONGS

The constructor initializes every data member before the constructor body begins. The parameter strings are received by const reference because the constructor only reads them.

```
Song::Song(const string& title, const string& artist, int durationSeconds)
    : title(title), artist(artist),  durationSeconds(durationSeconds),
      playCount(0) {
    ++numberOfSongs;
}
```

The order in the initializer list matches the declaration order. The static counter records the number of live Song objects rather than the number of songs registered in a particular library.

USING SONG OBJECTS

```
Song imagine("Imagine", "John Lennon", 183);
Song yesterday("Yesterday", "The Beatles", 125);
Song letItBe("Let It Be", "The Beatles", 243);
Song heyJude("Hey Jude", "The Beatles", 431);

imagine.play();
letItBe.play();
letItBe.play();
```

The four objects will remain part of the project. They are not temporary demonstrations. Albums, playlists, and the library will refer to these same objects later.

CONST CORRECTNESS INSIDE THE PROJECT

Printing a song does not change it, so print() is a `const` member function. Getters are also `const`. This allows Album, Playlist, and MusicLibrary to inspect Song objects through const references and const pointers.

```
void Song::print() const  {
    cout << title << " - " << artist << " (" << durationSeconds
         << " seconds, " << playCount << " plays)" << endl;
}
```

By contrast, play() changes playCount and therefore cannot be const.

COPYING A SONG WHEN THE APPLICATION NEEDS INDEPENDENCE

The project occasionally returns a Song by value. The returned object must be independent of the object stored in the library. The copy constructor performs that initialization.

```
Song::Song(const Song& other)
    : title(other.title),
      artist(other.artist),
      durationSeconds(other.durationSeconds),
      playCount(other.playCount)
{
    ++numberOfSongs;
}
```

No dynamic resource is owned by Song, so memberwise copying is appropriate. The copy receives the same descriptive values and play count, but it is a separate object with its own lifetime.

STEP 2: BUILDING ALBUM

An album groups songs that already exist. The Album object does not create or destroy them. It stores addresses of Song objects and therefore represents **aggregation**.

```
class Album
{
private:
    static const int MAX_SONGS = 12;
    string title;
    string artist;
    const Song* songs[MAX_SONGS];
    int songCount;

public:
    Album(const string& title, const string& artist);
    bool addSong(const Song& song);
    bool contains(const string& songTitle) const;
    Song getOpeningTrackCopy() const;
    void print() const;
};
```

The pointers are const Song* because an Album may inspect its songs but does not modify them. The Song objects remain independent and may also appear in playlists or in the library.

ADDING EXISTING SONGS TO AN ALBUM

```
bool Album::addSong(const Song& song)
{
    if (songCount == MAX_SONGS)
        return false;

    songs[songCount] = &song;
    ++songCount;
    return true;
}
```

The parameter is a const reference. No Song copy is created. The function stores the address of the original object and returns whether the operation succeeded.

```
Album beatlesCollection("Beatles Essentials", "The Beatles");
beatlesCollection.addSong(yesterday);
beatlesCollection.addSong(letItBe);
beatlesCollection.addSong(heyJude);
```

The relationship is now meaningful: one independent Song object can appear in the album and later in a playlist without being duplicated.

SEARCHING WITHIN AN ALBUM

```
bool Album::contains(const string& songTitle) const
{
    for (int i = 0; i < songCount; ++i)
    {
        if (songs[i]->getTitle() == songTitle)
            return true;
    }
    return false;
}
```

contains() is const because searching does not change the album. The function calls the const getter getTitle() through a pointer to const Song.

RETURNING A SONG BY VALUE

The application may ask an album for an independent copy of its opening track. Returning by value expresses that the caller receives a new Song rather than access to the album's internal relationship.

```
Song Album::getOpeningTrackCopy() const
{
    if (songCount == 0)
        return Song();

    return *songs[0];
}
```

Dereferencing songs[0] produces the existing Song object. Returning it by value initializes a separate result. The function never returns a reference to a local object and never exposes the internal pointer array.

STEP 3: BUILDING PLAYLIST

A playlist also aggregates existing songs, but its responsibility differs from Album. An album models a collection associated with an artist. A playlist models a user-selected ordering of songs.

```
class Playlist
{
private:
    static const int MAX_SONGS = 20;
    string name;
    const Song* songs[MAX_SONGS];
    int songCount;

public:

// The explicit keyword disables implicit conversions.
// Without explicit, a string could be converted automatically into
// a Playlist whenever a Playlist object is expected.
//
// Playlist p("Rock"); // OK
// Playlist p = "Rock"; // Error
    explicit Playlist(const string& name);
    bool addSong(const Song& song);
```

```
    bool contains(const Song& song) const;
    int totalDuration() const;
    void print() const;
};
```

The constructor is explicit because a string should not be converted into a Playlist accidentally. The class stores only non-owning pointers to songs that must remain alive while the playlist uses them.

PREVENTING DUPLICATE ENTRIES

```
bool Playlist::addSong(const Song& song)
{
    if (songCount == MAX_SONGS || contains(song))
        return false;

    songs[songCount] = &song;
    ++songCount;
    return true;
}
```

The helper function receives the candidate song by const reference and compares addresses. The playlist therefore rejects the same Song object if it is added twice.

```
bool Playlist::contains(const Song& song) const
{
    for (int i = 0; i < songCount; ++i)
    {
        if (songs[i] == &song)
            return true;
    }
    return false;
}
```

COMPUTING INFORMATION WITHOUT CHANGING THE PLAYLIST

```
int Playlist::totalDuration() const
{
    int total = 0;
    for (int i = 0; i < songCount; ++i)
        total += songs[i]->getDurationSeconds();
    return total;
}
```

The method is const. It reads the duration of each referenced Song and returns a calculated value. No member of Playlist or Song is modified.

```
Playlist favorites("Favorites");
favorites.addSong(imagine);
favorites.addSong(letItBe);
favorites.addSong(heyJude);
```

STEP 4: BUILDING MUSICLIBRARY

MusicLibrary coordinates the objects known to the application. In this version, it does not own their memory. It registers existing songs, albums, and playlists and provides operations that work across them.

```
class MusicLibrary
{
private:
    string ownerName;
    Song* songs[MAX_SONGS];
    Album* albums[MAX_ALBUMS];
    Playlist* playlists[MAX_PLAYLISTS];
    int songCount;
    int albumCount;
    int playlistCount;

public:
    explicit MusicLibrary(const string& ownerName);
    bool addSong(Song& song);
    bool addAlbum(Album& album);
    bool addPlaylist(Playlist& playlist);
    const Song* findSong(const string& title) const;
    Song getSongCopy(const string& title) const;
```

```
    void printSummary() const;
};
```

MusicLibrary stores Song* rather than const Song* because the registered songs may be played through other parts of the application. Its search function still returns const Song* to provide read-only access to callers.

REGISTERING OBJECTS

```
bool MusicLibrary::addSong(Song& song)
{
    if (songCount == MAX_SONGS ||
        findSong(song.getTitle()) != nullptr)
        return false;

    songs[songCount] = &song;
    ++songCount;
    return true;
}
```

The function receives Song& because the library stores the address of the original object. It does not copy the Song and does not take ownership of it.

```
MusicLibrary library("Elina");
library.addSong(imagine);
library.addSong(yesterday);
```

```
library.addSong(letItBe);
library.addSong(heyJude);
library.addAlbum(beatlesCollection);
library.addPlaylist(favorites);
```

SEARCHING THE LIBRARY

```
const Song* MusicLibrary::findSong(const string& title) const
{
    for (int i = 0; i < songCount; ++i)
    {
        if (songs[i]->getTitle() == title)
            return songs[i];
    }
    return nullptr;
}
```

The returned pointer refers to an existing Song. Returning nullptr represents a failed search. Because the return type is const Song*, callers may inspect the result but cannot use that pointer to modify the Song.

RETURNING AN INDEPENDENT COPY

```
Song MusicLibrary::getSongCopy(const string& title) const
{
    const Song* found = findSong(title);
    if (found == nullptr)
        return Song();

    return *found;
}
```

This operation serves a different purpose from findSong(). findSong() provides read-only access to an existing object. getSongCopy() creates a separate object that the caller may use independently.

```
Song localCopy = library.getSongCopy("Imagine");
localCopy.print();
```

THE COMPLETE APPLICATION

main() now connects the same objects introduced throughout the chapter. No example is discarded. Each statement contributes to the final application.

```
int main()
{
    Song imagine("Imagine", "John Lennon", 183);
    Song yesterday("Yesterday", "The Beatles", 125);
    Song letItBe("Let It Be", "The Beatles", 243);
    Song heyJude("Hey Jude", "The Beatles", 431);

    Album beatlesCollection("Beatles Essentials", "The Beatles");
    beatlesCollection.addSong(yesterday);
    beatlesCollection.addSong(letItBe);
    beatlesCollection.addSong(heyJude);

    Playlist favorites("Favorites");
    favorites.addSong(imagine);
    favorites.addSong(letItBe);
    favorites.addSong(heyJude);

    MusicLibrary library("Elina");
    library.addSong(imagine);
```

```
    library.addSong(yesterday);
    library.addSong(letItBe);
    library.addSong(heyJude);
    library.addAlbum(beatlesCollection);
    library.addPlaylist(favorites);

    imagine.play();
    letItBe.play();
    letItBe.play();

    library.printSummary();

    Song localCopy = library.getSongCopy("Imagine");
    localCopy.print();

    cout << Song::getNumberOfSongs() << endl;
    return 0;
}
```

REPRESENTATIVE OUTPUT

```
Playing: Imagine by John Lennon
Playing: Let It Be by The Beatles
Playing: Let It Be by The Beatles
Music Library for Elina
Songs: 4
Albums: 1
Playlists: 1

Album: Beatles Essentials by The Beatles
  1. Yesterday - The Beatles (125 seconds, 0 plays)
  2. Let It Be - The Beatles (243 seconds, 2 plays)
  3. Hey Jude - The Beatles (431 seconds, 0 plays)

Playlist: Favorites (3 songs, 857 seconds)
  1. Imagine - John Lennon (183 seconds, 1 plays)
  2. Let It Be - The Beatles (243 seconds, 2 plays)
  3. Hey Jude - The Beatles (431 seconds, 0 plays)
```

The same Song objects appear in several contexts. Playing letItBe changes its play count once, and the updated value is visible when both the album and playlist print that song. This confirms that these classes refer to the same independent object rather than owning separate copies.

OBJECT LIFETIME IN THE COMPLETE PROGRAM

The objects in main() are created in declaration order. The Album, Playlist, and MusicLibrary objects store addresses of Song objects that were created earlier. This order is deliberate: the referenced Song objects must remain alive while the aggregating objects use them.

When main() ends, local objects are destroyed in reverse order. localCopy is destroyed first, followed by library, favorites, beatlesCollection, and finally the four original Song objects. The aggregating classes do not delete the objects they reference.

This design works because all referenced objects live until the end of main(). A function must never return an Album or Playlist that contains pointers to local Song objects that have already been destroyed.

HOW UNIT 5 CONCEPTS APPEAR IN THE PROJECT

Concept	**Where It Appears**
Encapsulation	Each class keeps its state private and exposes purposeful operations.
Passing by value	Song is returned by value when an independent copy is required.
Passing by reference	MusicLibrary registers existing objects through references.
Passing by const reference	Album and Playlist receive songs without copying or modifying them.
Const member functions	Search, print, getters, and calculations are read-only operations.
Copy constructor	A returned Song initializes an independent local copy.
Initializer lists	Every constructor establishes complete initial state before its body.
Static members	Song counts all live Song objects.
Inline functions	Small getters are defined inside the class declaration.
Aggregation	Album, Playlist, and MusicLibrary refer to objects created elsewhere.
Lifetime	Referenced objects must outlive the objects that use them.

DESIGN DECISIONS AND THEIR CONSEQUENCES

The application deliberately separates identity from grouping. A Song is an independent entity. Albums and playlists do not duplicate it; they refer to it. As a result, a change to the original Song is observed wherever that Song appears.

The fixed capacities keep the project within the material covered so far. A production application would normally use standard containers and more advanced ownership techniques. Those changes would introduce additional decisions about copying, assignment, and resource management.

The project also distinguishes two forms of access. A function may return a const pointer when the caller should inspect an existing object, or return an object by value when the caller needs an independent copy. The correct choice depends on the intended relationship, not only on syntax.

COMMON PROJECT MISTAKES

• Storing the address of a temporary Song. The temporary is destroyed, leaving a dangling pointer.

• Creating Album or Playlist before the Song objects they will reference and then allowing the songs to leave scope early.

• Removing const from read-only parameters or member functions, which unnecessarily restricts how the classes may be used.

• Returning Song& from getSongCopy(). The operation promises independence, so it must return by value.

• Deleting aggregated objects in a destructor. These classes did not allocate the objects and therefore do not own them.

• Confusing the static count of live Song objects with the number of songs registered in one MusicLibrary.

SUMMARY

This unit combined the object-oriented techniques introduced so far into one coherent application. Song represents an independent object. Album and Playlist group existing songs for different purposes. MusicLibrary coordinates the objects known to the application.

- Class responsibilities were identified before implementation.
- Constructors and initializer lists established valid state.
- Const references and const member functions protected read-only operations.
- Pointers represented non-owning aggregation without dynamic allocation.
- Returning by value created an independent Song through copy construction.
- Static state counted all live Song objects.
- Object declaration order ensured that referenced objects remained alive.

REVIEW QUESTIONS

1. Why does Album store const Song* rather than Song objects?

2. Why does Album::addSong() receive const Song&?

3. What would change if Album stored Song objects by value?

4. Why is Playlist::totalDuration() declared const?

5. What does MusicLibrary::findSong() return when no matching song exists?

6. How does findSong() differ conceptually from getSongCopy()?

7. When is the Song copy constructor used in the complete application?

8. Why must the original Song objects outlive Album and Playlist?

9. Why should Album and Playlist not delete the Song objects they reference?

10. What does Song::getNumberOfSongs() count?

11. Why may the live Song count be larger than the number registered in MusicLibrary?

12. Which objects are destroyed first when main() finishes?

13. What risk would arise if a playlist stored the address of a local Song created inside another function?

14. Which project operations use const reference parameters?

15. Which design decisions would need to change if the program began allocating songs dynamically?

LOOKING AHEAD

In this unit, copying was safe because Song relied on string and ordinary integer data members. The compiler-generated memberwise copy produced independent, valid objects.

The situation changes when a class allocates memory dynamically and stores the address in a raw pointer. A simple memberwise copy then duplicates the address rather than the allocated resource. Two objects may accidentally share the same memory, leading to shallow copies, double deletion, and undefined behavior.

In the next unit, we will introduce dynamic memory inside classes, examine shallow and deep copying, implement the copy assignment operator, and bring the constructor, copy constructor, assignment operator, and destructor together through the Rule of Three.